



Bash Script

Database Management System

Ziad Osama & Youssef Amgad

**Telecom Application
Development**

Project Functional Requirements

The Script files

1. Project Application Script File (Main App)

Description

This script represents the **main application entry point** of the Bash-based DBMS.

It is responsible for managing databases at a high level and routing the user to database-level operations through the user menu script.

The application works on the concept that **each database is represented as a directory** inside the project directory.

Project Path: The project databases are stored under: \$HOME/Bash-Script-Linux-DBMS-Project/DBMS

Functions:

➤ **Create Database function**

Purpose: Create a new database represented as a directory inside the project path.

Flow:

1. Ask the user to enter the database name.
2. Validate that the name is not empty.
3. Check that no existing database directory has the same name.
4. If the name already exists, display an error message and terminate the function.
5. If the name is valid and unique:
 - Create a new directory with the database name.
 - Display a success message.

Validation Rules:

- Database name cannot be empty.
- Database name must be unique.

➤ **List Databases function**

Purpose: Display all existing databases in the system.

Flow:

1. Iterate over all directories inside the project path.
2. Display each directory name as a database with an index number.
3. Track whether at least one database exists.
4. If no database is found:
 - Display a message indicating that no databases exist.

Return Value:

- Returns 0 if at least one database exists.
- Returns 1 if no database is found (used by other functions for validation)

➤ **Connect to Database function**

Purpose: Allow the user to connect to an existing database and open the database user menu.

Flow:

1. Call the **List All Databases** function.
2. If no databases exist:
 - Display an error message and exit the function.
3. Ask the user to enter the database name to connect.
4. Validate that the name is not empty.
5. Check if the database directory exists.
6. If found:
 - Display a confirmation message.
 - Call MenuUser.sh script.
 - Pass the following arguments:
 1. Project Path
 2. Database Name
7. If not found:
 - Display an error message.

Arguments Passed to User Menu Script:

- Project directory path
- Connected database name

➤ Drop Database function

Purpose: Delete an existing database from the system.

Flow:

1. Call the **List All Databases** function.
2. If no databases exist:
 - Display an error message and exit the function.
3. Ask the user to enter the database name to be deleted.
4. Validate that the name is not empty.
5. Search for a matching database directory.
6. If found:
 - Remove the database directory recursively.
 - Display a success message.
7. If not found:
 - Display an error message indicating the database does not exist.

Validation Rules:

- Database name cannot be empty.
- Database must exist before deletion.

Main Application Menu:

Menu Options

1. Create Database
2. List Databases
3. Connect To Database
4. Drop Database
5. Exit Application

Behavior

- Each option triggers the corresponding function.
- Invalid options are handled with an error message.
- Exit option terminates the application gracefully with a farewell message.

Notes

- Each database is implemented as a directory.
- Validation is applied before any critical operation.
- The application ensures separation between:
 - Database-level management (Main App)
 - Table-level operations (User Menu Script)

2. The Menu User script file

Description

This script represents the **database-level user interface**.

It is called after connecting to a specific database and allows the user to perform all **table-level operations** inside the selected database.

Each table is represented as a **text file** inside the database directory.

Arguments

- \$1 → Project Path
- \$2 → Database Name

The full database path is constructed as:

DBPath = ProjectPath/DBName

Helper Functions:

➤ Table Existing Status Function:

Purpose: Check whether a specific table exists inside the connected database.

Arguments: Table Name

Flow:

1. Validate that the table name is not empty.
2. Search for a file with the same name inside the database directory.
3. Return: 0 if the table exists and 1 if the table does not exist or the name is empty.

Functions

➤ **Create Table Function:**

Purpose: Create a new table inside the connected database.

Flow:

1. Ask the user to enter the table name.
2. Validate that the name is not empty.
3. Check if the table already exists.
4. If the table exists:
 - Display an error message.
5. If the table does not exist:
 - Create a new empty file to represent the table.
 - Display a success message.
 - Call **CreatTable.sh** File.

Arguments Passed:

- Table Path
- Table Name
- Database Name

➤ **List All Tables Function:**

Purpose: Display all tables inside the connected database.

Flow:

1. Iterate over all files inside the database directory.
2. Display each table name with an index number.
3. If no tables exist:
 - Display a message indicating that no tables are found.
4. Return a status code to indicate whether tables exist.

➤ **Drop Table Function:**

Purpose: Delete an existing table from the database.

Flow:

1. Call **List All Tables** function.
2. If no tables exist:
 - Display an error message and exit the function.
3. Ask the user to enter the table name.
4. Validate that the name is not empty.
5. Check if the table exists.
6. If found:
 - Remove the table file.
 - Display a success message.
7. If not found:
 - Display an error message.

➤ **Insert Data into Table Function:**

Purpose: Insert new records into a selected table.

Flow:

1. Call **List All Tables** function.
2. Ask the user to enter the table name.
3. Validate the table name.
4. Check if the table exists.
5. If found:
 - Call **InsertDataInTable.sh** File.

Arguments Passed:

- Table Path
- Table Name
- Database Name

➤ **Update Data in Table Function:**

Purpose: Update existing records in a table.

Flow:

1. Call **List All Tables** function.
2. Ask the user to enter the table name.
3. Validate the table name.
4. Check if the table exists.
5. If found:
 - Call **UpdateDataInTable.sh** File.

Arguments Passed:

- Table Path
- Table Name
- Database Name

➤ **Select From Table Function:**

Purpose: Retrieve and display data from a table.

Flow:

1. Call **List All Tables** function.
2. Ask the user to enter the table name.
3. Validate the table name.
4. Check if the table exists.
5. If found:
 - Call **SelectDataFromTable.sh** File.

Arguments Passed:

- Table Path
- Table Name
- Database Name

➤ **Delete From Table Function:**

Purpose: Delete records from a table.

Flow:

1. Call **List All Tables** function.
2. Ask the user to enter the table name.
3. Validate the table name.
4. Check if the table exists.
5. If found:
 - Call **DeleteDataFromTable.sh** File.

Arguments Passed:

- Table Path
- Table Name
- Database Name

➤ **Alter Table Structure Function:**

Purpose: Modify the structure of an existing table.

Flow:

1. Call **List All Tables** function.
2. Ask the user to enter the table name.
3. Validate the table name.
4. Check if the table exists.
5. If found:
 - Call **AlterStructureTable.sh** File.

Arguments Passed:

- Table Path
- Table Name
- Database Name

User Menu

Description: The script runs inside an infinite loop that provides the database user with table-level operations.

Menu Options

1. Create New Table
2. List All Tables
3. Drop Table
4. Insert Data into Table
5. Update Data in Table
6. Select From Table
7. Delete From Table
8. Alter Structure Table
9. Back to Main Application

Behavior

- Each menu option triggers its corresponding function.
- Input validation is applied for all user inputs.
- Selecting **Back** exits the user menu and returns control to the main application.

Notes

- Tables are stored as text files.
- All table operations are isolated per database.
- Script files are modular to ensure maintainability and readability.
- Strong validation is applied before file operations.

3. Create Table Script File:

Description

This script is responsible for **creating the structure of a table** inside a specific database.

It handles:

- Defining the **Primary Key**
- Adding table columns with validation
- Managing column data types
- Updating existing records when new columns are added

Each table is represented as a **text file**, where:

- The **first line** contains column definitions.
- Columns are separated by (:) delimiter.
- Each column is stored in the format: ColumnName(DataType):

Arguments

- \$1 → Table Path
- \$2 → Table Name
- \$3 → Database Name

Functions

➤ **Add New Column Function:**

Purpose: Add a new column to the table with full validation and update existing records accordingly.

Flow:

1. Column Name Validation

1. Ask the user to enter the column name.
2. Validate that the column name:
 - Is not empty.
 - Is not numeric.
 - Follows valid naming conventions:
 - Starts with a letter or underscore.
 - Contains only letters, numbers, and underscores.
3. Check that the column name does not already exist in the table header.
4. If validation fails:
 - Display an error message.
 - Repeat the process until valid input is provided.

2. Column Data Type Selection

1. Display a menu of supported data types:
 - int
 - float
 - string
 - char
2. Ask the user to choose the data type.
3. Validate the selected option.
4. Allow the user to cancel the process.

3. Update Table Structure

1. Append the new column definition to the table header.
2. Update the first line of the table file with the new header.
3. For all existing records:
 - Append a default value (NULL) for the newly added column.

Notes:

- Existing records are preserved.
- Default values are added automatically to maintain consistency.

Primary Key Definition

Description

Before adding any additional columns, the script enforces defining a **Primary Key (PK)**.

Flow:

1. Prompt the user to enter the primary key column name.
2. Validate that the PK:
 - Is not empty.
 - Is not numeric.
 - Contains only alphabetical characters.
3. Automatically assign:
 - Data type: int
4. Append the PK definition to the table header.
5. Prevent proceeding until a valid PK is defined.

Constraints:

- Each table must have exactly one primary key.
- The primary key is always of type int.

Main Menu

Description: After setting the primary key, the script provides a menu to manage the table structure.

Menu Options

1. Add New Column
2. Back

Behavior

- **Add New Column:**
 - Calls the Add New Column function.
- **Back:**
 - Exits the script and returns control to the database user menu.
- Invalid inputs are handled with appropriate error messages.

Notes

- Column definitions are stored in the first line of the table file.
- Strong validation ensures:
 - No duplicate column names.
 - Valid column naming conventions.
- The script maintains data integrity by updating existing records.
- The design supports incremental table structure building.

4. Insert Data into Table Script File:

Description

This script is responsible for **inserting records into a specific table** inside a database.

It ensures:

- Data type validation
- Primary key constraints
- Data consistency with the table structure

Each record is stored as a single line in the table file, with fields separated by (:) delimiter.

Arguments

- \$1 → Table Path
- \$2 → Table Name
- \$3 → Database Name

Helper Functions

➤ **List Columns of Table Function:**

Purpose: Display all columns of the table with their data types and store clean column names for internal use.

Flow:

1. Read the first line (header) of the table file.
2. Split the header by (:) delimiter.
3. Extract column names without data types.
4. Display each column with its index and full definition.
5. Store column names in a global array for later processing.

➤ **Check Data Type Function:**

Purpose: Determine the data type of user input dynamically.

Supported Types:

- int
- float
- char
- string

Flow:

1. Match the input value against regular expressions.
2. Return the detected data type.
3. Used for validating user input against column data types.

Functions

➤ **Insert Data Function**

Purpose: Insert a new record into the selected table while enforcing table constraints.

Insert Process Flow

1. Read Table Structure

- Read the table header.
- Extract column names and their data types.
- Initialize an empty row buffer to store the new record.

2. Primary Key Insertion

Constraints:

- Primary Key is always the **first column**.
- Must be:
 - Not empty
 - Unique
 - Matching the defined data type

Flow:

1. Prompt the user to enter a value for the primary key.
2. Validate that the value is not empty.
3. Validate the data type.
4. Check uniqueness by scanning existing records.
5. Repeat until valid input is provided.

3. Insert Data into Other Columns

Flow:

1. For each remaining column:
 - Ask the user whether to insert data (Y/N).
2. If the user chooses **Yes**:
 - Prompt for the value.
 - Validate the data type.
3. If the user chooses **No**:
 - Insert NULL as the default value.
4. Repeat validation until correct input is entered.

4. Save the Record

- Append the constructed row to the table file.
- Ensure field order matches column order.
- Display a success message.

Main Menu

Description: The script runs inside a loop that provides insert-related operations for the selected table.

Menu Options

1. Insert Data into Table
2. List Columns of Table
3. Back

Behavior

- **Insert Data:**
 - Calls the Insert Data function.
- **List Columns:**
 - Displays table structure and column definitions.
- **Back:**
 - Returns control to the database user menu.

Notes

- Records are stored line by line after the header.
- Data integrity is preserved through strict validation.
- Primary key uniqueness is enforced at insertion time.
- Default NULL values are used when data is not provided.
- Script is modular and reusable across tables.

5. Delete Data From Table Script File:

Description

This script is responsible for **displaying and deleting records from a specific table** inside a database. It provides options to:

- Display all table data
- Delete all rows
- Delete rows based on specific column values

The table data is stored in a text file where:

- The **first line** represents the table header (column definitions).
- Each subsequent line represents a record.
- Fields are separated using (:) delimiter.

Arguments

- \$1 → Table Path
- \$2 → Table Name
- \$3 → Database Name

Functions

➤ **Display All Rows Function:**

Purpose:

Display all table data in a formatted and readable table layout.

Flow:

1. Display a header title for the table.
2. Read the entire table file.
3. Format and align the output using `column -t` with `:` as a delimiter.
4. Pause execution until the user presses Enter.

➤ **Delete Rows Function:**

Purpose: Provide deletion operations for table records. 'Delete Rows Menu'

Menu Options

1. Delete All Rows
2. Delete Rows by Column Values
3. Back

Delete All Rows Functionality

Flow:

1. Read the table header (first line).
2. Overwrite the table file with only the header.
3. Remove all existing records.
4. Display a success message.

Constraint:

- Column definitions are preserved.

Delete Rows by Column Values Functionality

Flow:

1. Read the table header.
2. Extract column names (without data types).
3. Display available column names.
4. Ask the user to:
 - Enter the target column name.
 - Enter one or more values (space-separated).
5. Validate that the column exists.
6. Create a temporary file.
7. Copy the header to the temporary file.
8. Use pattern matching to:
 - Skip rows where the target column matches any of the given values.
9. Replace the original table file with the filtered temporary file.
10. Display a success message.

Validation Rules:

- Column name must exist in the table.
- Only rows matching the specified values are removed.

Main Menu

Description: The script runs inside an infinite loop that allows the user to manage deletion operations.

Menu Options

1. Display Table
2. Delete Rows
3. Back

Behavior

- **Display Table:**
 - Calls the Display All Rows function.
- **Delete Rows:**
 - Opens the delete rows submenu.
- **Back:**
 - Returns control to the database user menu.

Notes

- Deletion operations never remove the table structure.
- Temporary files are used to ensure safe deletion.
- Output is formatted for better readability.
- The script supports deleting multiple records in a single operation.

6. Select Data From Table Script File:

Description

This script is responsible for **retrieving and displaying data from a specific table** inside a database. It provides flexible selection operations including:

- Selecting all data
- Selecting specific columns
- Selecting rows based on column values

All outputs are formatted in a readable tabular form using column alignment.

Arguments

- \$1 → Table Path
- \$2 → Table Name
- \$3 → Database Name

Functions

➤ **Display All Function:**

Purpose: Display the full contents of the table including all columns and rows.

Flow:

1. Print a formatted title for the table.
2. Read the entire table file.
3. Display data in a table format using `column -t` with `( : )` delimiter.
4. Pause execution until the user presses Enter.

➤ **Select Rows Function:**

Purpose: Allow the user to retrieve records based on specific criteria. 'Select Rows Menu'

Menu Options

1. Display All Rows
2. Display Rows By Column Values
3. Back

Display All Rows Functionality

Flow:

1. Display all table data in formatted layout.
2. Wait for user confirmation before returning.

Display Rows by Column Values Functionality

Flow:

1. Read the table header.
2. Extract and display available column names.
3. Ask the user to:
 - Select a column to filter by.
 - Enter one or more values (space-separated).
4. Validate that the selected column exists.
5. Retrieve and display only rows where the column matches any of the provided values.
6. Display the filtered result in a formatted table.

Validation Rules:

- Selected column must exist in the table.
- At least one value must be provided.

➤ **Select Columns Function:**

Purpose: Allow the user to select and display only specific columns from the table.

Flow:

1. Display all available column names.
2. Ask the user to enter column names separated by spaces.
3. Validate column existence (case-insensitive).
4. Display only the selected columns for all records.
5. Format the output as a table.
6. Allow the user to return to the main menu.

Main Menu

Description: The script provides a main selection menu that allows navigating between different data retrieval operations.

Menu Options

1. Select All Columns
2. Select Specific Columns
3. Select Rows
4. Back

Behavior

- Each option triggers the corresponding selection function.
- Invalid inputs are handled gracefully.
- Back option returns control to the database user menu.

Notes

- Selection operations never modify table data.
- Temporary files are used for filtering operations when needed.
- Output is always aligned for better readability.
- Script supports multi-value filtering and partial data views.

7. Update Data In Table Script File:

Description

This script is responsible for **updating existing records in a specific table** inside a database. It provides multiple update mechanisms while preserving:

- Data type integrity
- Primary key constraints
- Table structure consistency

The script supports updating:

- A single record by **Primary Key**
- Multiple records by **column value**

Arguments

- \$1 → Table Path
- \$2 → Table Name
- \$3 → Database Name

Helper Functions

➤ **List Columns of Table Function:**

Purpose: Display all columns of the table with their data types and store clean column names for validation.

Flow:

1. Read the table header.
2. Split the header using (:) delimiter.
3. Extract and display column names with their data types.
4. Store column names (without data types) in a global array.

➤ **Check Data Type Function:**

Purpose: Detect and validate the data type of user input.

Supported Types:

- int
- float
- char
- string

Flow:

1. Match the input value against predefined patterns.
2. Return the detected data type.

Functions

➤ **Update Data by Primary Key Function:**

Purpose: Update a single record identified uniquely by the primary key.

Update By PK Flow

1. Check whether the table contains data records.
2. Read table header and extract column definitions.
3. Ask the user to enter the primary key value.
4. Validate:
 - Value is not empty.
 - Data type matches the PK column type.
 - PK value exists in the table.
5. For each non-PK column:
 - Display the current value.
 - Ask the user whether to update it.
 - Validate new value data type.
 - Update the value in-place using record position.
6. Save changes back to the table file.
7. Display success messages for each updated column.

Constraints:

- Primary key value itself cannot be modified.

➤ **Update Data by Column Value Function:**

Purpose: Update one or more records based on a specific column value.

Flow

1. Display all table columns.
2. Ask the user to enter the column name.
3. Validate:
 - Column exists.
 - Column is not the primary key.
4. Ask the user to enter:
 - Old value to search for.
 - New value to replace with.
5. Validate data types for both values.
6. Check if the old value exists in the column.
7. Update all matching records.
8. Save the updated table content.

Constraints:

- Primary key column cannot be updated.
- Multiple rows may be updated in one operation.

➤ **Show Table Function:**

Purpose: Display the entire table content in a formatted view.

Flow:

1. Read the table file.
2. Display the data using column alignment.
3. Pause execution until the user presses Enter.

Main Menu

Description: The script runs inside a loop providing update-related operations.

Menu Options

1. Update Data by Primary Key
2. Update Data by Value of Column
3. List Columns of Table
4. Show Table
5. Back

Behavior

- Each option triggers its corresponding function.
- Validation is applied before any update operation.
- Back option returns control to the database user menu.

Notes

- Data updates are performed safely using temporary files.
- Data type validation ensures consistency.
- Primary key uniqueness and immutability are enforced.
- Script supports both single-row and multi-row updates.

8. Alter Table Structure Script File:

Description

This script is responsible for **modifying the structure of an existing table** inside a database.

It allows the user to:

- Add new columns
- Drop existing columns
- Rename columns

The script ensures:

- Primary Key protection
- Data consistency
- Safe modification of table structure and records

Arguments

- \$1 → Table Path
- \$2 → Table Name
- \$3 → Database Name

Helper Functions

➤ **List Columns of Table Function:**

Purpose: Display all columns of the table with their data types and store clean column names for validation and indexing.

Flow:

1. Read the first line (header) of the table.
2. Split column definitions using (:) delimiter.
3. Extract column names without data types.
4. Display each column with its index.
5. Store column names in a global array.

Functions:

➤ **Add New Column Function:**

Is the same in [CreateTable.sh](#) file.

➤ **Drop Column Function:**

Purpose: Remove an existing column from the table.

Flow:

1. Display all table columns.
2. Ask the user to enter the column name.
3. Validate:
 - Column name is not empty.
 - Column exists in the table.

4. Prevent dropping the **Primary Key** column.
5. Remove the column from:
 - Table header
 - All existing records
6. Save the updated table structure.

Constraints:

- Primary Key column cannot be dropped.
- Column must exist before deletion.

➤ **Rename Column Function (Alter Column):**

Purpose: Rename an existing column without modifying its data.

Flow:

1. Display all table columns.
2. Ask the user to enter the column name to rename.
3. Validate column existence.
4. Ask the user to enter the new column name.
5. Validate that the new name is not empty.
6. Update only the table header.

Special Case:

- If the column is the **Primary Key**:
 - Only renaming is allowed.
 - Data type and data remain unchanged.

Main Menu

Description: The script runs inside an infinite loop that allows the user to manage table structure operations.

Menu Options

1. Add New Column
2. Drop Column
3. Rename Column
4. List Columns of Table
5. Back

Behavior

- Each menu option calls its corresponding function.
- Input validation is applied for all operations.
- Back option returns control to the database user menu.

Notes

- Table structure is stored only in the first line of the table file.
- All operations preserve existing data integrity.
- Primary Key is fully protected:
 - Cannot be dropped
 - Can only be renamed
- Script uses safe file rewriting to prevent data corruption.